

Módulo V “Desarrolla aplicaciones para dispositivos móviles”

Submódulo I “Desarrolla aplicaciones móviles para Android”.

--- Parte 2 ---

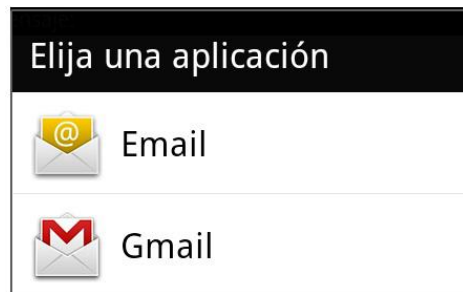
INTENT IMPLÍCITO

En el **intent implícito** no nombramos específicamente el componente que va a ejecutar esa acción que queremos que realice. Lo que permite que otro componente que sea capaz de realizar esa acción la pueda ejecutar.

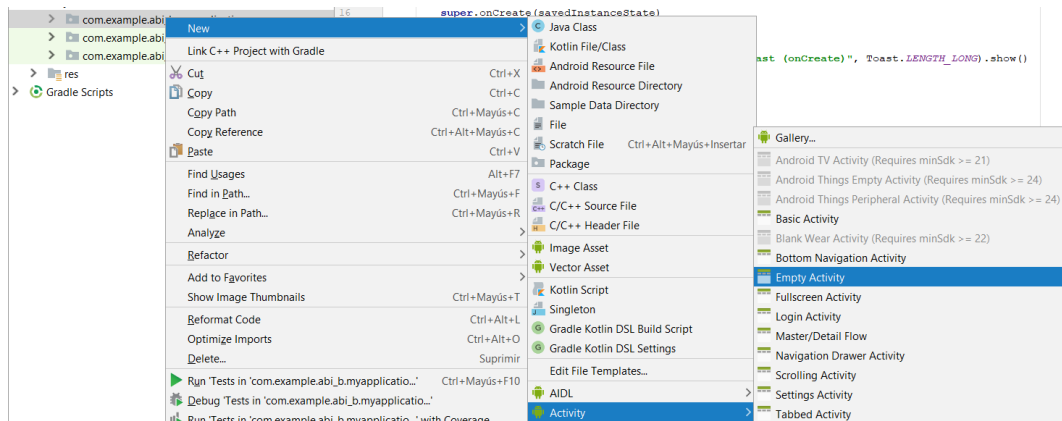
Por ejemplo, si creamos una app donde le permitimos al usuario enviar emails, con el intent implícito declaramos una acción general para que el usuario realice dicha operación, pero no nombramos qué componente va a llevar acabo esta acción, por lo que cuando el usuario le ponga “enviar”, va a salir un mensaje de selección donde pedirá elegir una opción (proveedor) que nos enviará el email (email, Gmail, Outlook, etc.).

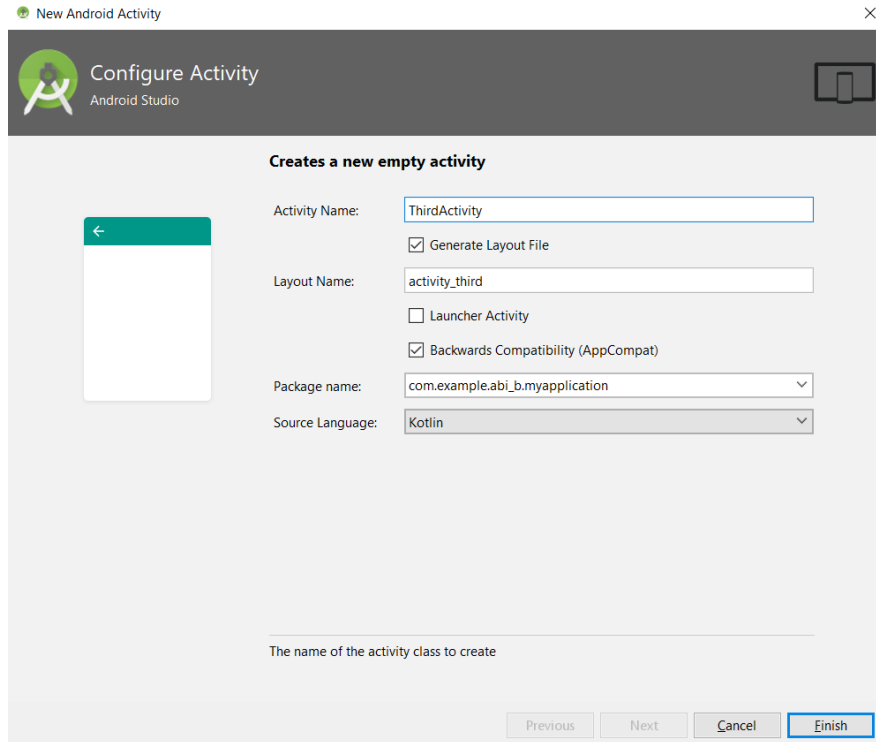
Otro ejemplo es el “Compartir este video vía..”, ¿con qué opciones podemos compartirlo? Vía bluetooth, Gmail, WhatsApp, Mensaje, etc., teniendo varias opciones para que se pueda llevar a cabo dicha acción.

Se declara dicha acción general, pero no nombramos qué componente la llevará acabo. Siguiendo con los ejemplos, cuando queremos visitar una página web, nos sale un recuadro “Completa la acción usando...”, seleccionando el navegador predeterminado, Chrome, Mozilla, etc.



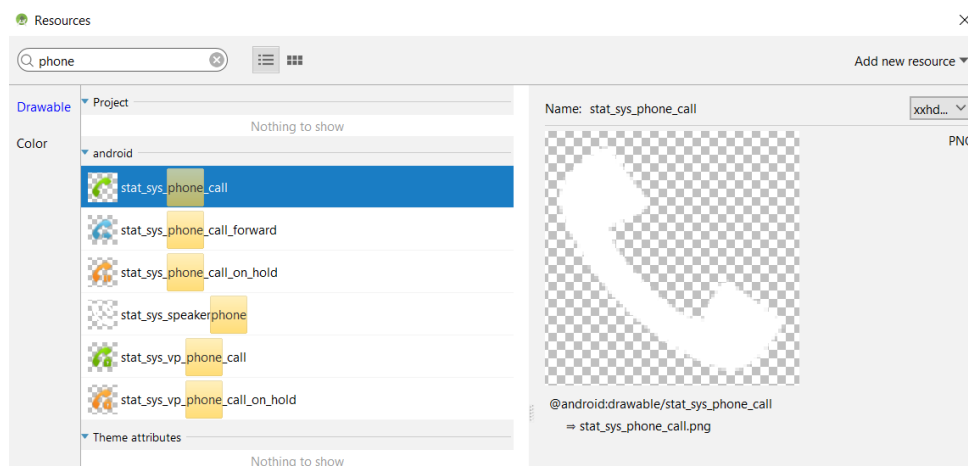
Ahora creamos en nuestro paquete (aplicación), un activity donde ya tenemos los demás, al cual nombraremos ThirdActivity.



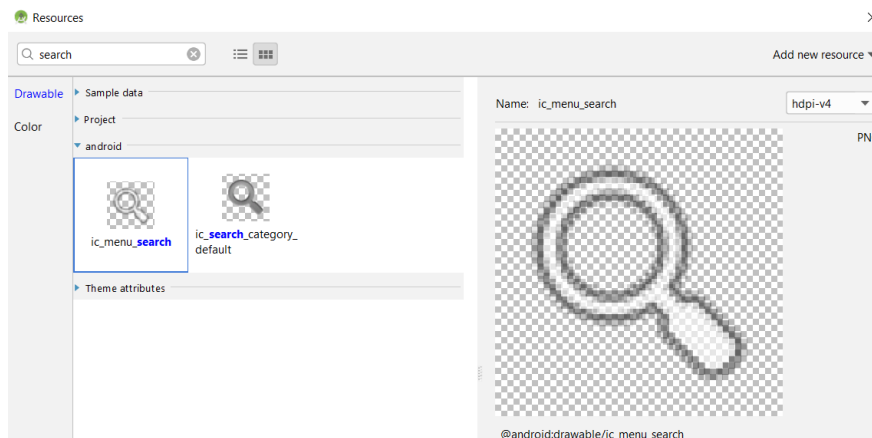


La idea de esta aplicación es que cuando el usuario ingrese un número telefónico, estaremos trabajando con distintos intents (con el intent del teléfono), siendo capaces de que dicho número ingresado pueda enlazar una llamada, haremos también que ingrese una dirección de una página web para que cuando presione un botón, le permita al usuario que la app lo redirija a esa página que busca (intent web). Además habrá varios botones, uno de ellos será para saber cómo crear correos electrónicos, otro botón para realizar una llamada diferente a la antes mencionada y otro botón para que accedamos a la cámara del dispositivo.

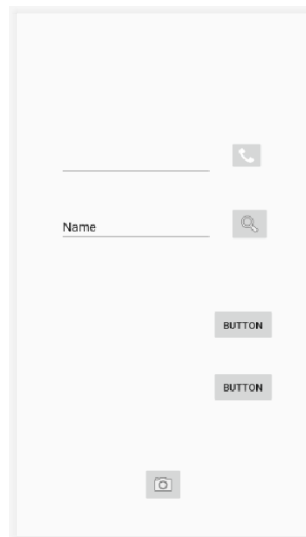
Seguimos trabajando con el mismo proyecto, pero ahora estaremos en el layout que acabamos de crear (activity_third.xml, Design). Agregaremos un Phone, un Plain Text y un ImageButton. Al añadir este último, nos aparece una ventana para elegir qué imagen tendrá dicho botón (buscamos "phone").



Agregamos otro ImageButton, y en la ventana buscaremos “search” e incluimos el ícono de la lupa.



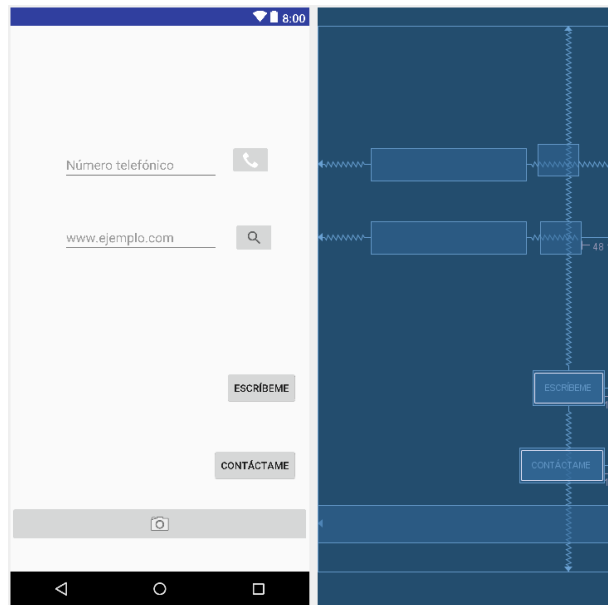
Agregamos 2 botones, y un ImageButton (buscamos “camera”) dejando el diseño que se muestra a continuación:



Pasamos al .xml para ir cambiando algunas propiedades de los elementos que acabamos de agregar. Empezamos con los EditText (Phone y Plain Text), al primero le cambiamos el id por editTextPhone y le agregamos la propiedad hint, “Número telefónico”. En el siguiente EditText, cambiamos el id por editTextWeb, eliminamos la propiedad text y agregamos la propiedad hint, “www.ejemplo.com”.

Pasamos a los ImageButton, al primero le cambiamos el id por ImageButtonPhone, y al siguiente por ImageButtonWeb.

Ahora trabajaremos con los 2 botones. Al primero, en el id, lo nombraremos buttonEmailMe y en su propiedad text, “Escríbeme”. Al siguiente, lo nombramos buttonContactPhone y “Contáctame”. En el último ImageButton, cambiamos la propiedad id por ImageButtonCamera, y en la propiedad layout_width escribimos match_parent.



Nos vamos al código .xml para rediseñar los ImageButton. En el primer ImageButton colocaremos la propiedad background, quedando de la siguiente forma:

`android:background="@android:color/transparent"`.

Además, con esta propiedad, vemos que el ícono de teléfono desaparece, por lo que modificamos la siguiente propiedad:

`app:srcCompat="@android:drawable/sym_action_call"`.

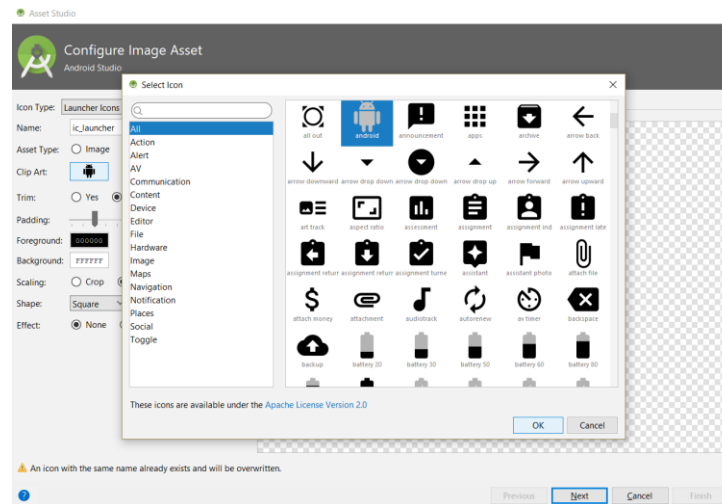
En ImageButtonWeb, agregamos la propiedad background de la siguiente forma:

`android:background="@android:color/transparent"`.

Por último, hacemos clic en para que lo infiera por nosotros, quedando de la siguiente forma:



Rediseñaremos los botones, agregando un ícono al botón. Para esto, nos vamos a **File -> New -> Image Asset**. En la opción Icon Type, seleccionamos Launcher Icons (Legacy Only) y hacemos clic en el ícono de Android (Clip Art).



En la ventana que se abre, buscamos "email", y le cambiamos la propiedad Name y Shape, a ic_email y None, respectivamente. Hacemos clic en Next y Finish. Hacemos el mismo procedimiento, pero ahora buscamos "phone" y seleccionamos phone_android. Le cambiamos la propiedad Name y Shape, a ic_phone y None, respectivamente. Hacemos clic en Next y Finish. Con esto, ya tenemos creados nuestros iconos, ¿pero en dónde se almacenan? Se guardan en una carpeta dentro de la de recursos (app -> res -> mipmap).

Para poder agregar esos iconos recién creados, nos vamos al código .xml, y buscamos el botón buttonEmailMe y agregamos la siguiente línea:

```
android:drawableLeft="@mipmap/ic_email"
```

donde drawableLeft indica que el ícono estará al principio de la palabra. Además, le agregamos un espaciado para que quede mejor distribuido, con la siguiente instrucción:

```
android:drawablePadding="10dp"
```

Se observa un espacio y el padding significa que le va a dar un margen interior de 10dp. Agregamos un paddingEnd, para que el texto Escríbeme se separe del final del botón

```
android:paddingEnd="19dp"
```

Nos vamos al botón buttonContactPhone y usamos las mismas propiedades que en el anterior botón, drawableLeft, llamando al ícono ic_phone, agregamos un paddingEnd de 12dp y un drawablePadding de 10dp.

En dado caso, de que el ImageButtonCamera quede separado de la parte de abajo, lo seleccionamos y en los atributos referente a los márgenes, colocamos un 0.

Además, les agregamos las propiedades textColor y background-Tint para manipular el color del texto y el fondo del botón, respectivamente.

Pasamos a la implementación de nuestra lógica en el archivo **ThirdActivity.kt**.

La idea es que cuando el usuario le dé clic al botón recuperará el número que ya haya ingresado en el editText. Para esto, creamos lo siguiente: lo primero a implementar es la llamada. Accedemos al image-ButtonPhone mediante las siguientes líneas:

```
binding.imageButtonPhone.setOnClickListener(object: View.OnClickListener{
    override fun onClick(v: View?) {
        val phoneNumber = binding.editTextPhone.text.toString()
        if (!phoneNumber.isEmpty()){
            val intentCall = Intent(Intent.ACTION_DIAL, Uri.parse("tel:$phoneNumber"))
            startActivity(intentCall)
        }
        else
            Toast.makeText(this@ThirdActivity, "Debes marcar un número, intenta nuevamente",
                Toast.LENGTH_LONG).show()
    }
})
```

Lo primero en hacer es una manera diferente de cómo asignar un evento onClick. Empezamos a escribir el `setOnClickListener`, seguido de `object`, el cual recibe una vista y nos autocompleta con el `View.OnClickListener`. Seguimos dentro de los paréntesis y escribimos `{ y }`; dentro de éstos, escribimos `override fun onClick` para sobrescribir la función `onClick`. Creamos una variable tipo `val` llamada `phoneNumber` siendo igual al `binding.editTextPhone.text.toString()` para acceder al contenido que tenga.

Hacemos una comprobación con `if (!phoneNumber.isEmpty())`, es decir, si la variable `phoneNumber` no está vacía, entonces entramos a la estructura condicional.

¿Cómo le decimos que va a realizar una acción de llamada? Mediante el `Intent.ACTION_DIAL`, seguido de `Uri.parse`, añadiéndole una referencia, siendo `"tel"` (comportamiento), y la variable `phoneNumber`. Después de esto, ya podemos realizar un `startActivity()` del `intentCall`. Pero a todo esto, ¿qué es un uri? **Un uri** lo podemos asociar a la idea de lo que es una url en internet. Cuando ponemos un uri, estamos pasándole un identificador para que sepa cómo se comportará esta acción. Por ejemplo, cuando queremos acceder a alguna página de internet, empezamos por `http://www.ejemplo.com`, en el caso del uri, es `tel: 8341440958`, es decir, estamos indicando qué rol jugará en el protocolo.

Por lo que `tel` es el identificador para la llamada y deberá comportarse de tal forma. En Android, los uri se utilizan para apuntar hacia otras aplicaciones o para apuntar a acciones, como en este ejemplo, se intenta ejecutar una acción de una llamada.

Para terminar, hacemos otro `else` en donde comprobamos si el número que esté llegando no está vacío, escribiendo lo siguiente:

```
else{
    Toast.makeText(this@ThirdActivity,"Debes marcar un número, intenta nuevamente",
        Toast.LENGTH_LONG).show()
}
```

Posteriormente hacemos la llamada para el `imageButtonWeb` junto con su método `setOnClickListener`. Creamos la variable `url` y será igual a lo que el usuario ingrese en el `editTextWeb`. Además, creamos una instancia `intentWeb`, pero trabajaremos ahora de forma diferente. Ya se aprendió la manera en que le pasábamos directamente la acción y el uri, pero se puede desarrollar sin parámetros, de la siguiente manera.

Declaramos `intentWeb.action` para pasar la acción que estará esperando, siendo igual a `Intent.ACTION_VIEW`, y la instrucción que actuará como uri es `intentWeb.data = Uri.parse("https://$url")`. Después de esto, podemos lanzar nuestro `startActivity` con el `intentWeb`.

El código del `imageButtonWeb`, queda de la siguiente manera:

```
binding.imageButtonWeb.setOnClickListener{
    val url = binding.editTextWeb.text.toString()
```

```

        val intentWeb = Intent()
        intentWeb.action = Intent.ACTION_VIEW
        intentWeb.data = Uri.parse("https://$url")
        startActivity(intentWeb)
    }

```

Algo muy importante, en este proyecto debemos agregar el permiso para conexión a internet, ya que estamos usando el intent para la web, el cual se concede en el AndroidManifest.xml.

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Empezamos haciendo el evento click de buttonEmailMe.setOn ClickListener, creando una variable email, siendo igual a un correo x (miemail@gmail.com, para ejemplificar); creamos una instancia de la clase intent, siendo igual a Intent(Intent.ACTION_SENDTO, Uri.parse(email)), donde ACTION_SENDTO es la acción esperada para enviar el correo, y un uri que será nuestra variable email.

Terminamos lanzando el intent mediante el startActivity. La actividad del buttonEmailMe queda de la siguiente forma:

```

binding.buttonEmailMe.setOnClickListener {
    val mailto = "mailto:ejemplo@hotmail.com" + "?cc=" + "ejemplo2@gmail.com" +
        "&subject=" + Uri.encode("Asunto del correo") + "&body=" + Uri.encode("Esta es una prueba...")
    val emailIntent = Intent(Intent.ACTION_SENDTO, Uri.parse(mailto))
    try {
        startActivity(Intent.createChooser(emailIntent, "Elige el cliente de correo..."))
    } catch (e: ActivityNotFoundException) {
        Toast.makeText(this, "No hay ningún cliente de correo electrónico instalado.",
            Toast.LENGTH_LONG).show()
    }
}

```

Ahora seguimos con la implementación del evento clic del botón de la llamada (buttonContactPhone). Creamos la instancia intentCall siendo igual a Intent(Intent.ACTION_DIAL, Uri.parse("tel: 8341440958")), donde ACTION_DIAL es una llamada directa sin permisos, es más rápida porque no se verifica ningún permiso, y el Uri.parse es un teléfono cualquiera y marcará en cuanto se oprima el botón; terminamos llamando al intent creado mediante el startActivity.

El código completo queda de la siguiente manera:

```

binding.buttonContactPhone.setOnClickListener {
    val intentCall = Intent(Intent.ACTION_DIAL, Uri.parse("tel: 2288302966"))
    startActivity(intentCall)
}

```

Siguiendo con la app, ahora nos enfocaremos en el imageButtonCamera, el cual accederá a la cámara del dispositivo, por lo tanto, trabajaremos en el ThirdActivity.kt, quedando de la siguiente manera:

```

binding.imageButtonCamera.setOnClickListener {
    val intentCamera = Intent("android.media.action.IMAGE_CAPTURE")
    startActivity(intentCamera)
}

```

Lo siguiente será que el ícono o logotipo se mantenga en el action bar. Trabajaremos en el MainActivity.kt. ¿Y qué es el **action bar**? Una barra de herramientas principal dentro de la actividad que puede mostrar el título de la actividad, los recursos de navegación a nivel de la aplicación y otros elementos interactivos.



Esto se hace mediante las siguientes líneas:

```
supportActionBar!!.setDisplayHomeAsUpEnabled(true)  
supportActionBar!!.setIcon(R.mipmap.ic_launcher)
```

El método `setIcon` fuerza la aparición del ícono en el action bar, donde ya se tiene precargado el `ic_launcher`, dejando a criterio si se quiere crear, modificar o cargar dicho ícono, adjuntándolo en la carpeta `mipmap` o `drawable`. El resultado es el siguiente:



Además, si queremos cambiarle el ícono que visualiza el usuario a la app creada, nos iremos al `AndroidManifest.xml` y se cambia en la propiedad `android:icon = "@mipmap/ic_launcher"`, donde `ic_launcher` es el ícono que viene cargado por default; si se desea cambiar el nombre de la app, la propiedad encargada de esto es `android:label="@string/app_name"`, y si presionamos `Ctrl + clic`, nos abrirá el archivo `strings.xml` donde está la propiedad `app_name`, y lo cambiamos por el nombre que se desea se visualice al instalar la aplicación.

En el `AndroidManifest.xml` se almacenan todos los activities que se van generando, y observamos que el `.MainActivity` tiene un `intent-filter`, el cual indica qué activity se va a lanzar como principal al momento de abrir la app.